Dave Deonarain

EEE4701

October 13, 2026

Synchronous FIFO Verification & Validation Report

Demo Video: https://youtu.be/9ttGHkSJhwU

1. **Executive Summary and Verification Goals**

   This report should detail the verification functionality, coverage and mutation of synchronous FIFO. The verification was constructed in Python using cocotb, pyvsc, cocotb-coverage, and Verilator.

   The primary objective of this project was to establish a testbench while bin coverage is closed in the verification plan and testing its efficiency in craft_mutate. This verification is achieved by 100% coverage in all bins, reaching a mutation score of 90% or more, and zero regression and clean RTL.

   Using design parameters of WIDTH = 8 and DEPTH = 8, using inputs rsn_n, wr_en, rd_en, wr_data and outputs rd_data, full, empty, count, these should all be synchronous to the active edge of clk with a 10 ns period. The data should be visible on rd_data when the buffer is not empty and require reading.

   From the verification plan, it should include targets on reset recovery, concurrent read/write transaction, burst pointer wrapping and boundary overflow/underflow protection.

2. **Testbench Architecture & Functional Coverage**

   The structured architecture of the environment separates drive, monitoring and scoreboard. This is the list below:

   - FifoDriver (tb/driver.py): Drives control of the falling clock edge (FallingEdge(clk)). This brings a half period setup margin prior to the rising edge, avoiding any race conditions.
   - FifoMonitor (tb/monitor.py): This initialize a zero time evaluation routine, sample_now(), that samples the state of the falling edge of the pin. This shows the simulator time advance.
   - FifoScoreboard (tb/scoreboard.py): This uses a model algorithm and pointer counters to evaluate hardware outputs cycle by cycle. This should always check for count consistency, flag logic and payloads.
   - FifoSeqItem (crv/fifo_item.py): This uses pyvsc for constrained random transacton.

The cocotb-coverage model also defines 68 bins. Below, bullet points the bins for this model.

- Control Strobes: wr_en and rd_en toggling have 2 bins each
- Occupancy Bins: Counts 0 to 8 have 9 discrete bins
- Flag State Bins: empty, full and intermediate non-empty (3 legal states)
- Cross Products: 4 command combinations (cross_cmd), 36 command by occupancy combinations (cross_cmd_x_count), and 12 command by flag combinations (coss_flags_x_cmd)

Random uniform stimulus was fixed at 62.5% because random read/write draws would linger near mid-range occupancy. To close these boundaries these test sequences was structured into four stages:

- From 0 to 8, all four command pairs at each occupancy are swept direct linearly.
- Full and sustained reads at empty when boundary soak tests hold
- Alternate bursts write and drain sweeps stressing pointer wraparound
- 2,000 cycle constrained random stress phase using pyvsc

The progression of this structure brought 100% coverage where all bins close.

**3. Mutation Testing & Residual Risk**

Mutation testing was executed via craft_mutate against mutants_released.yaml. Below is the command used for this execution.
“PYTHONPATH=../../lib python -m craft_mutate.mutate --tb . --pool mutants_released.yaml”.

The test killed all 8 mutants. Below is a brief description of each.

- full_offbyone: Asserted full at count 7. Caught by the scoreboard during fill sweeps when checking sample.full == (count == 8).
- empty_offbyone: Asserted empty at count 1. Caught by scoreboard checks asserting empty == 0 following first write.
- ignore_full: Authorized writes when full. Caught during boundary soaks by verifying count saturation and order preservation.
- ignore_empty: Authorized reads when empty. Caught by verifying count remained at 0 and pointer did not decrement.
- count_wr_dec: Decremented count on writes. Caught right away on the first write cycle by count checks against the model.
- rd_data_wrptr: Indexed read data from the write pointer. Caught by data comparison on the initial read.
- mem_corrupt: Corrupted stored data (+1) on write. Caught by scoreboard read data checks against expected values.

- wrptr_wrap_early: Wrapped the write pointer one slot early. Caught during burst sequences writing past slot 7.

Originally, the scoreboard assertion caught 5 out of 8 mutants. Adding directed sweeps, this increased to 7 out of 8 and for boundary soak test paired with falling edge sampling, it found 8 out of 8 totals .

Verification risks includes asynchronous reset pulse mid-cycle, setup/hold timing margin degradation from physical clock jitter, and streaming latency mismatches. Iterations in the future would benefit from binding concurrent SystemVerilog Assertions (fifo_sva.sv) directing into internal pointers.

4. **"Check the Machine" AI Reflection & Contributions**

As per guidelines in the project, using AI code generations was auditied through prediction, testing and correction. Below is how exactly it contributed in completing this project.

- **Transaction Generation** (crv/fifo_item.py): AI output prediction was to fail the SystemVerilog types with pyvsc. As expected, execution threw "AttributeError: module 'vsc' has no attribute 'rand_uint_t'. This was resolved by switching to vsc.rand_but_t() and applying range constraints.
- **Coverage Modeling** (crv/coverage.py): AI output was predicted to reference non existent helper methods, mishandle ranges, and trigger sampling race conditions. Simulation failed with "AttributeError: 'CoverageDB' object has no attribute 'summary' " where bins stalled coverage at 62.5%, and post-clock sampling produced scoreboard mismatches. By iterating over "coverage_db.items(), expanding occupancy to explicit integer bins [0…8], and incorporating zero time falling edge sampling by sample_now().

This entire project was done solo where I implemented a testbench driver, monitor, and scoreboard. It was configured with constrained random transaction generation and closed all 68 coverage bins. And when executing mutation, it has achieved 8/8 kills recorded in commit. This includes writing a vplan.md, FINDINGS.md, ai_log.md files as well as a short demo video going over the project structure and the results.